



Top 20 RAG Interview Questions

20 Questions Every AI Engineer Should Know

CHECK **CAPTION** FOR **AI INTERVIEW** BUNDLE

What is Retrieval Augmented Generation (RAG), and why is it critical for LLMs?

RAG lets a language model look up information before answering. A normal LLM only knows what it learned during training; RAG adds a search step before the answer is generated. An LLM without RAG is a student answering from memory; with RAG, the student can open a book, read the right page, and then answer. In a RAG system the model: (1) receives a question, (2) searches a database or documents for relevant information, (3) uses that information to generate a final answer. RAG is critical because it allows LLMs to use up-to-date information, answer from private or company data, and reduce wrong or made-up answers—making answers more accurate, trustworthy, and useful.

CHECK **CAPTION** FOR **AI INTERVIEW** BUNDLE



What are the limitations of a pure LLM system without RAG?

A pure LLM can only rely on what it already knows. First, knowledge is frozen in time—if the model was trained last year, it does not know what happened yesterday. Second, it cannot access private data (company policies, internal documents, customer records). Third, it may hallucinate: confidently giving an answer that sounds correct but is wrong, like someone explaining a document they have never seen. Without RAG, LLMs are limited to general knowledge, old information, and higher risk of incorrect answers.

CHECK **CAPTION** FOR **AI INTERVIEW** BUNDLE



I have created Ultimate AI Engineer Interview Bootcamp

12 Courses | Structured Learning | Interview Mastery



**PYTHON
INTERVIEW
ESSENTIALS**

REGISTER

Python Essentials for AI Engineer



**STATISTICS & MATH
FOR AI/ML
INTERVIEWS**

REGISTER

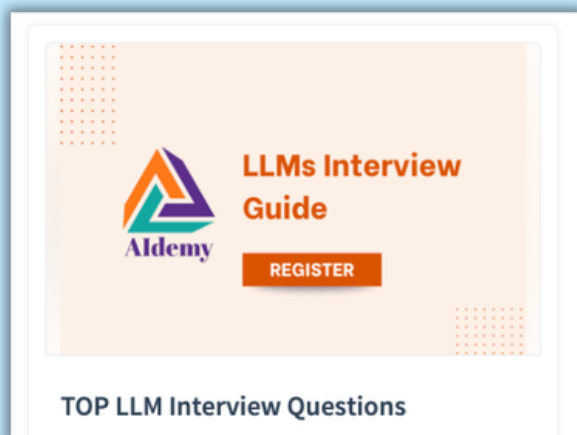
Statistics & Math for AI/ML Interviews



**PROMPT
ENGINEERING
MASTERY**

REGISTER

Prompt Engineering Interview Mastery



**LLMs Interview
Guide**

REGISTER

TOP LLM Interview Questions



**LANGCHAIN
INTERVIEW
MASTERY**

REGISTER

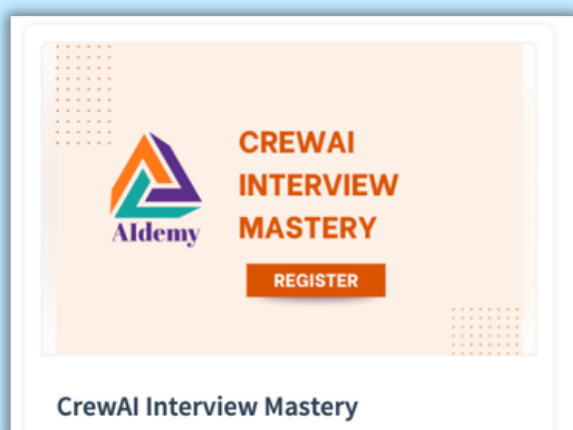
LangChain Interview Mastery



**LANGGRAPH
INTERVIEW GUIDE**

REGISTER

LangGraph Interview Preparation Gu...



**CREWAI
INTERVIEW
MASTERY**

REGISTER

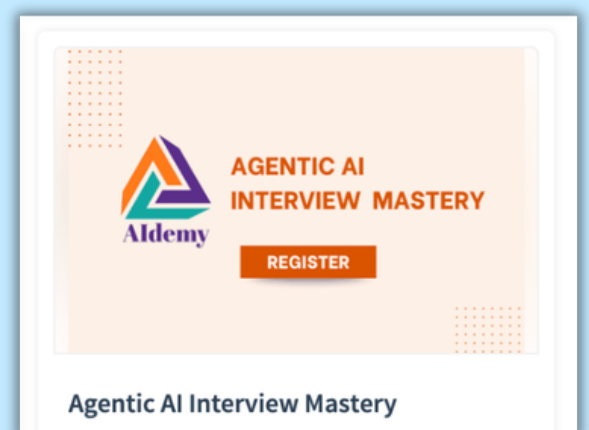
CrewAI Interview Mastery



**AUTOGEN
INTERVIEW
PREPARATION**

REGISTER

AutoGen Interview Preparation



**AGENTIC AI
INTERVIEW MASTERY**

REGISTER

Agentic AI Interview Mastery

ENROLL NOW

*Link in caption

How is RAG different from fine-tuning an LLM?

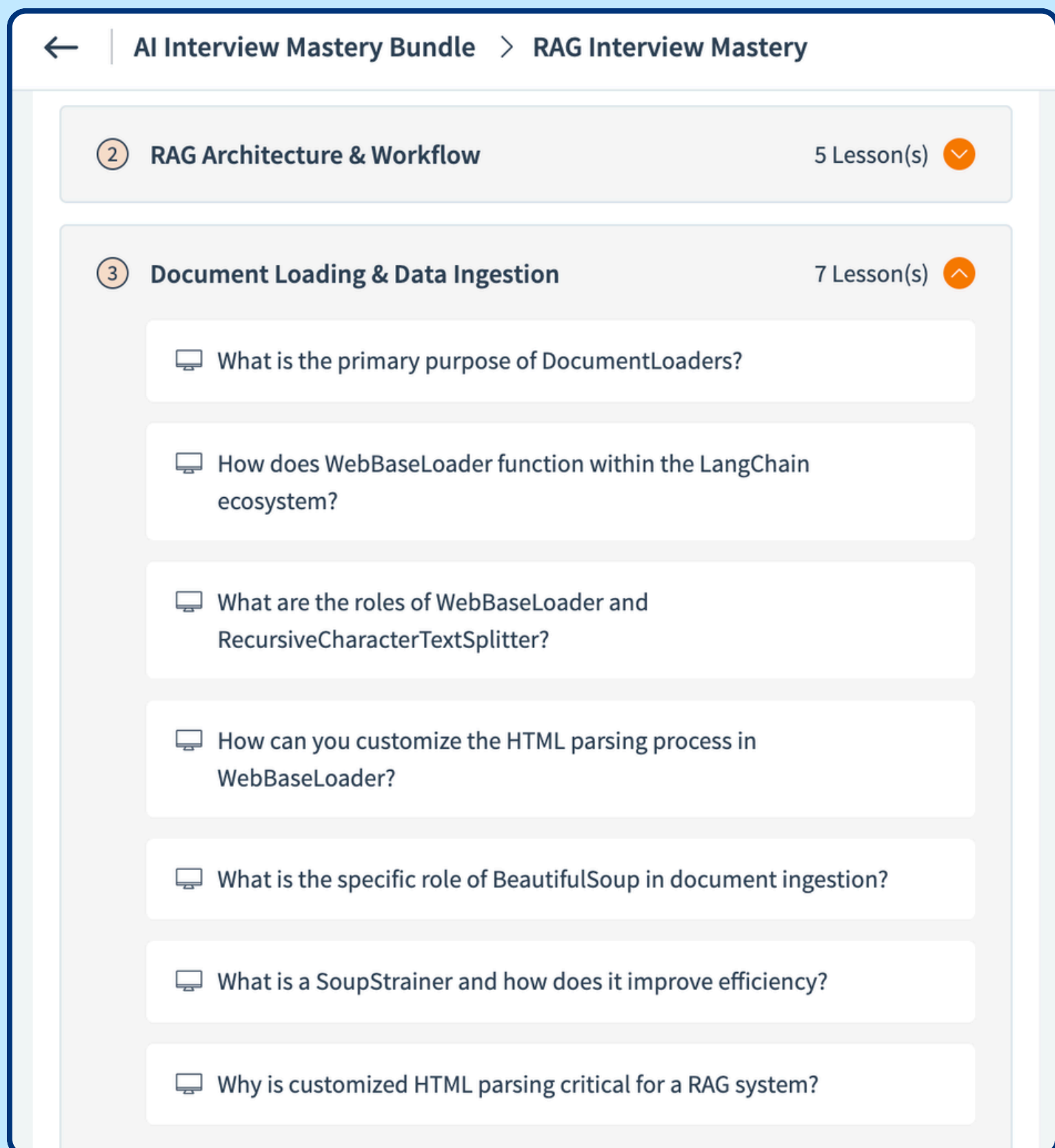
Fine-tuning means changing the model itself by training it on new examples. RAG keeps the model the same but gives it extra information at runtime. Fine-tuning is like teaching a student new subjects permanently; RAG is like letting the student use reference books during the exam. With fine-tuning, knowledge is baked in and updating requires retraining (slower, more expensive). With RAG, knowledge lives outside the model, you can update documents anytime, and the model always reads the latest data. RAG focuses on knowledge access; fine-tuning on behavior and style.

CHECK **CAPTION** FOR **AI INTERVIEW** BUNDLE



Each course divided into section wise questions and answers

12 Courses | Structured Learning | Interview Mastery



ENROLL NOW

*Link in caption

When should you prefer RAG over fine-tuning?

Prefer RAG when your data changes often, is large, or is private/company-specific—e.g. product documentation, internal policies, legal/medical guidelines, customer support knowledge bases. RAG is also better when you want faster updates, lower cost, and better transparency (you can see the source of answers). Fine-tuning is better when you want to change tone or writing style, teach special rules, or improve performance on a specific task. In many real systems, RAG is used first; fine-tuning is added only if needed.

CHECK **CAPTION** FOR **AI INTERVIEW** BUNDLE



What types of problems are best solved using RAG?

RAG works best for knowledge-based problems where the correct answer already exists in documents. Examples: question answering over documents, customer support chatbots, enterprise search, legal and compliance systems, technical documentation assistants. If the task requires looking things up, quoting policies, or explaining stored information, RAG is a great fit. RAG is less useful for creative writing, pure reasoning puzzles, or tasks that do not rely on external knowledge.

CHECK **CAPTION** FOR **AI INTERVIEW** BUNDLE



What kinds of data sources can be used in a RAG system?

A RAG system can use PDFs, Word documents, text files, web pages, databases, wikis, knowledge bases, and emails/chat logs. Documents are usually: (1) broken into small pieces, (2) converted into embeddings (numerical representations), (3) stored in a vector database. When a question comes in, the system finds the most relevant pieces and sends them to the LLM, so the model answers based on your data—that is what makes RAG powerful in real-world applications.

CHECK **CAPTION** FOR **AI INTERVIEW** BUNDLE



What are the two main components of a RAG application architecture?

(1) Retrieval component: searches for relevant information from external knowledge sources (documents, PDFs, databases, web pages). (2) Generation component: uses an LLM to generate a final answer by combining the user's question with the retrieved information. Example: for "What is the company leave policy?" the retrieval component fetches relevant HR documents; the generation component reads them and produces a clear, human-like answer.

CHECK **CAPTION** FOR **AI INTERVIEW** BUNDLE



How does the indexing process work, and why is it essential?

Indexing prepares documents so they can be searched efficiently. Steps: (1) documents are split into small chunks (paragraphs or sections), (2) each chunk is converted into an embedding, (3) embeddings are stored in a vector database for fast similarity search. Indexing is essential because LLMs cannot directly search large document collections; without it, finding relevant information would be slow and inaccurate. Example: a 200-page PDF is broken into sections stored as embeddings; when a question is asked, the system quickly finds the most relevant sections.

CHECK **CAPTION** FOR **AI INTERVIEW** BUNDLE



What is the step-by-step workflow of a complete RAG pipeline?

(1) User query—the user asks a question. (2) Query embedding—the question is converted into an embedding. (3) Retrieval—the system searches the vector database for relevant document chunks. (4) Ranking and selection—the most relevant chunks are selected. (5) Context construction—retrieved content is combined with the user's question. (6) Generation—the LLM generates a final answer using this combined context. Example: user asks "Explain transformer attention"; the system retrieves relevant textbook sections and generates a clear explanation.

CHECK **CAPTION** FOR **AI INTERVIEW** BUNDLE



What is the specific role of the Retrieval component?

The Retrieval component finds accurate, relevant information from external data sources based on the user's question. It ensures the LLM receives fact-based context, the response is grounded in real data, and the risk of hallucinations is reduced. Example: in a medical chatbot, retrieval fetches information from verified medical guidelines before the LLM generates a response.

CHECK **CAPTION** FOR **AI INTERVIEW** BUNDLE



What is the goal of integrating retrieval with generation?

The goal is to combine factual accuracy with natural language generation. Retrieval provides correct and up-to-date information; generation turns that into clear, readable answers. Instead of the LLM guessing from memory, it reads the actual document and explains it in simple language. This results in more accurate answers, fewer hallucinations, better trustworthiness, and easier updates without retraining the model.

CHECK **CAPTION** FOR **AI INTERVIEW** BUNDLE



What is the primary purpose of DocumentLoaders?

DocumentLoaders bring content from different places into LangChain in a standard format. They take data from websites, PDFs, Word documents, text files, databases and convert it into LangChain's Document format: page_content (extracted text) and metadata (URL, file name, page number). Example: for a RAG chatbot, one policy in a PDF and another on a website can both be loaded into the same format so the rest of the pipeline works smoothly.

CHECK **CAPTION** FOR **AI INTERVIEW** BUNDLE



How does WebBaseLoader function within the LangChain ecosystem?

WebBaseLoader is a DocumentLoader that loads content from a web page URL. It (1) takes a URL (e.g. a blog or docs page), (2) downloads the HTML, (3) extracts readable text from that HTML, (4) returns the text as one or more LangChain Document objects. You give a URL; WebBaseLoader fetches the page and gives you the main text content, which you can then chunk and store in a vector DB for RAG.

CHECK **CAPTION** FOR **AI INTERVIEW** BUNDLE



What are the roles of WebBaseLoader and RecursiveCharacterTextSplitter?

They work in two stages. WebBaseLoader brings the raw content from the web page into LangChain.

RecursiveCharacterTextSplitter breaks that large text into smaller chunks. Example: a 10,000-character article—

WebBaseLoader loads it as one Document;

RecursiveCharacterTextSplitter splits it into chunks (intro, steps, examples, FAQs). Embeddings and retrieval work best with smaller chunks, not huge text blocks.

CHECK **CAPTION** FOR **AI INTERVIEW** BUNDLE



How can you customize the HTML parsing process in WebBaseLoader?

You can customize what gets extracted by telling the loader which part of the page to read and which to ignore, usually via BeautifulSoup settings (e.g. selecting only main content). If you load a page directly you might get menu text, footer, ads. The goal is to extract only real content like the `<article>` or `<main>` section.

CHECK **CAPTION** FOR **AI INTERVIEW** BUNDLE



What is the specific role of BeautifulSoup in document ingestion?

BeautifulSoup helps you read and understand HTML structure. Web pages are full of tags (div, nav, footer); BeautifulSoup parses these tags, helps select useful parts, and helps extract clean text. Example: it can extract only the text inside `<main>` or `<article>` and ignore everything else.

CHECK **CAPTION** FOR **AI INTERVIEW** BUNDLE



What is a SoupStrainer and how does it improve efficiency?

A SoupStrainer tells BeautifulSoup: "Don't parse everything. Parse only this part." This improves efficiency because parsing becomes faster and you avoid extracting noisy content. Example: instead of parsing the whole page, parse only the <article> section so embeddings are built only from meaningful content, not menus and footers.

CHECK **CAPTION** FOR **AI INTERVIEW** BUNDLE



Why is customized HTML parsing critical for a RAG system?

RAG quality depends on what you store in the vector database. If you store noisy text ("Sign up for newsletter", "Home | About | Contact", "Cookie settings"), retrieval will bring those chunks back and the LLM will produce poor answers. Customized parsing ensures the vector DB contains only real article content, documentation steps, and important headings—improving retrieval accuracy, answer quality, and reducing hallucinations.

CHECK **CAPTION** FOR **AI INTERVIEW** BUNDLE



Why is it necessary to split documents into smaller chunks?

Documents are split so the model can understand, store, and retrieve information more effectively. Large documents often mix many topics; if kept as one piece, embeddings become vague and retrieval inaccurate. Example: a 50-page policy document has leave, payroll, holidays, insurance—one big chunk may retrieve payroll when the question is about leave. Splitting ensures each chunk focuses on one idea at a time.

CHECK **CAPTION** FOR **AI INTERVIEW** BUNDLE



Why must long documents be split before embedding?

Long documents must be split because embedding models have token limits, and very long text produces weak or averaged embeddings. Embedding a huge document dilutes meaning because the vector tries to represent too many ideas at once. Example: embedding a full textbook chapter gives a "general" vector; embedding a single paragraph gives precise meaning, which improves similarity search.

CHECK **CAPTION** FOR **AI INTERVIEW** BUNDLE



FOLLOW ME



Chandra Sekhar

For more AI & ML content
